Understanding Random Forests: A Practical Tutorial on Attribute Bagging and Variance Reduction

Author: Deshan Chathuranga Herath Pathiranage

Student ID: 24178639

Github Repository:
https://github.com/Deshan-Chathuranga/Random_Forest_Algorithm_With_Attribute_Bagging.git

Course: 7PAM2021-0105-2025 - Machine Learning and Neural Networks

---

1. Introduction

One of the hardest things to do in machine learning is to make models that work well on data that hasn't been seen before. A lot of algorithms, especially Decision Trees, tend to overfit by picking up on noise instead of useful patterns. This leads to models that do well on training data but not so well when they are used in real life. To make predictive systems that work, we need to deal with this problem.

This tutorial is all about **Random Forests**, a strong way to learn that combines many Decision Trees to get around their problems. The main idea behind this tutorial is **attribute bagging**, which adds randomness by picking a random set of features at each split. This process makes trees less correlated, which makes the model more stable and accurate.

The main goal of this tutorial is to teach students how Random Forests work in both theory and practice. This tutorial doesn't just focus on business uses; it also talks about the basic ideas behind machine learning, such as ensemble learning, feature randomness, and variance reduction.

A customer churn dataset is used as a real-world example to help people learn. The dataset gives a real-world example of how Random Forests and attribute bagging work. Students will learn how this method makes predictions better than a single Decision Tree by using the algorithm step by step.

- After completing this tutorial, readers will be capable of:
- Recognize Decision Trees' limitations
- Describe how overfitting is reduced by Random Forests.
- Explain how attribute bagging enhances model performance.
- Use Python to implement a Random Forest model.

2. Understanding the Data: The Foundation of Prediction

The dataset used in this tutorial is a customer churn dataset containing demographic, behavioural, and service-related attributes. Each row represents a customer, and the target variable indicates whether the customer churned (1) or was retained (0).

https://www.kaggle.com/datasets/miadul/customer-churn-prediction-business-dataset/data

2.1 Dataset Description

Key features include:

- **Tenure Months:** Duration of customer relationship
- **Monthly Logins:** Indicator of user engagement
- **CSAT Score:** Customer satisfaction rating (1–5)
- **Support Tickets:** Number of issues reported
- **Contract Type:** Monthly or long-term subscription
- **Churn:** Target variable

Understanding these features is crucial because machine learning models rely heavily on the quality and relevance of input data. For instance, low engagement combined with high complaint frequency is often a strong indicator of churn.

2.2 The "Churn Signature" and Exploratory Data Analysis (EDA)

In real life, we see that customers who leave often have a signature that can be seen. Statistically, a customer with a lot of support tickets and a low CSAT score is four times more likely to leave. We show why we need non-linear models like Random Forests by using histograms and correlation matrices to show these distributions. Simple linear correlations often miss these complicated interactions between multiple variables.

3. The Challenge: Overfitting in Decision Trees

1. Decision Trees are a common choice for classification tasks because they are easy to understand and use. They work by repeatedly dividing the data into groups that are all the same based on the values of the features.
2. But Decision Trees have a big problem: they have a lot of variation. They often pick up noise instead of useful patterns because they try to perfectly fit the training data. This leads to overfitting, which means that the model doesn't do well on new data.
3. A small change in the dataset can change the tree structure completely, which makes predictions less reliable. This shows that we need a stronger approach.

3.1 The Structure of a Decision Tree

Recursively dividing the feature space into hierarchical branches is how a decision tree works. The algorithm chooses the feature and threshold that minimizes **Gini Impurity** at each node.

Gini Impurity Formula:

$$G = 1 - \Sigma (P_i^2)$$

where Pi is the likelihood that an object will fall into a specific class. Single trees have high variance, despite being intuitive and simple to visualize. A tree tries to isolate every data point because it is "greedy" and makes the best local split at each step. Predictions are unreliable when applied to new cohorts because even a slight alteration in the training data can produce an entirely different tree structure (Breiman, 1996).

The Gini Impurity quantifies the likelihood that a randomly selected element from the set would have an incorrect label if it were randomly assigned a label based on the subset's label distribution. Within our churn dataset:

- G = 0 if a node is "pure" (all 100 customers in that branch stayed, for example).

- G = 0.5 if a node is perfectly mixed (50 stayed, 50 churned).Finding splits that bring this value closer to zero is the Decision Tree's objective.

3.2 Mathematical Mechanics: From Entropy to Information Gain

Although Gini is the default, it is crucial to comprehend Entropy and Information Gain. The "disorder" in the churn data is measured by entropy (H). The decrease in entropy brought about by data partitioning is known as information gain.

$$H(S) = — \Sigma [Pi * \log_2(Pi)]$$

A single tree frequently discovers hyper-specific rules that are exclusive to the training set in high-dimensional churn datasets (e.g., "Customers in London with 3 tickets who joined on a Tuesday churned"). By restricting the tree's "vision" at each split, attribute bagging purposefully prevents the tree from discovering these noisy rules.

4. The Ensemble Solution: Evolution of Bagging

Ensemble learning is the main tactic used to address variance. If a single tree is an unreliable "expert," a group of trees constitutes a "committee."

4.1 Bias–Variance Tradeoff

Examining the bias–variance tradeoff is crucial to comprehending Random Forests' efficacy.

- Errors resulting from overly simplistic assumptions are referred to as **Bias** (underfitting).
- **Sensitivity** to minute changes in data is referred to as variance (overfitting).
- The data contains inherent **irreducible noise**.

Generally speaking, decision trees have high variance but low bias. The objective is to lower variance without appreciably raising bias, which is where ensemble approaches come into play.

5. Ensemble Learning and Bagging

Several models are combined in ensemble learning to generate predictions that are superior to those made by any one model.

5.1 Bootstrap Aggregating (Bagging)

By training several models on various subsets of the data, bagging is an ensemble technique that lowers variance.

Steps involved:

1. Make several datasets by utilizing replacement sampling.
2. For every dataset, train a different model.

3. Use majority voting to combine forecasts.

This method lessens overfitting and increases stability. Standard bagging has a drawback, though: if a feature is very predictive, all models will likely rely on it, producing trees that are similar.

6. Attribute Bagging: The Core Innovation

Attribute bagging, sometimes referred to as the random subspace method, is the main innovation in Random Forests.

The algorithm chooses a **random subset of features** at each split rather than taking into account every feature. The square root of the total number of features is usually used to determine how many features are taken into account.

6.1 Why Attribute Bagging Works

Without attribute bagging:

- Trees start to resemble each other.
- There is a correlation between predictions.
- Has small variance reduction.

With attribute bagging:

- Trees diversify
- Various features are investigated
- Model gets stronger.

Because combining uncorrelated models results in more precise and reliable predictions, this diversity is crucial.

7. Random Forest Algorithm

Both bagging and attribute bagging are combined in a Random Forest:

1. Create multiple bootstrap samples.
2. With every sample, train a decision tree.
3. Choose a random subset of features at each split.
4. Combine all of the trees' predictions.

Out-of-Bag (OOB) samples are also used by Random Forests for validation. They can be used to estimate model performance without a separate validation set because some data points are excluded from each bootstrap sample.

8. Implementation

8.1 Data Preprocessing

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

df = pd.read_csv('customer_churn_business_dataset.csv')

df['complaint_type'] = df['complaint_type'].fillna('None')

le = LabelEncoder()
X = df.drop(columns=['customer_id', 'churn'])
y = df['churn']

for col in X.select_dtypes(include=['object']).columns:
    X[col] = le.fit_transform(X[col])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

8.2 Model Training

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    max_features='sqrt',
    random_state=42
)

model.fit(X_train, y_train)
```

8.3 Model Evaluation

```
from sklearn.metrics import roc_auc_score

y_pred_prob = model.predict_proba(X_test)[:, 1]
auc = roc_auc_score(y_test, y_pred_prob)

print("ROC-AUC Score:", auc)
```

9. Effect of Number of Trees

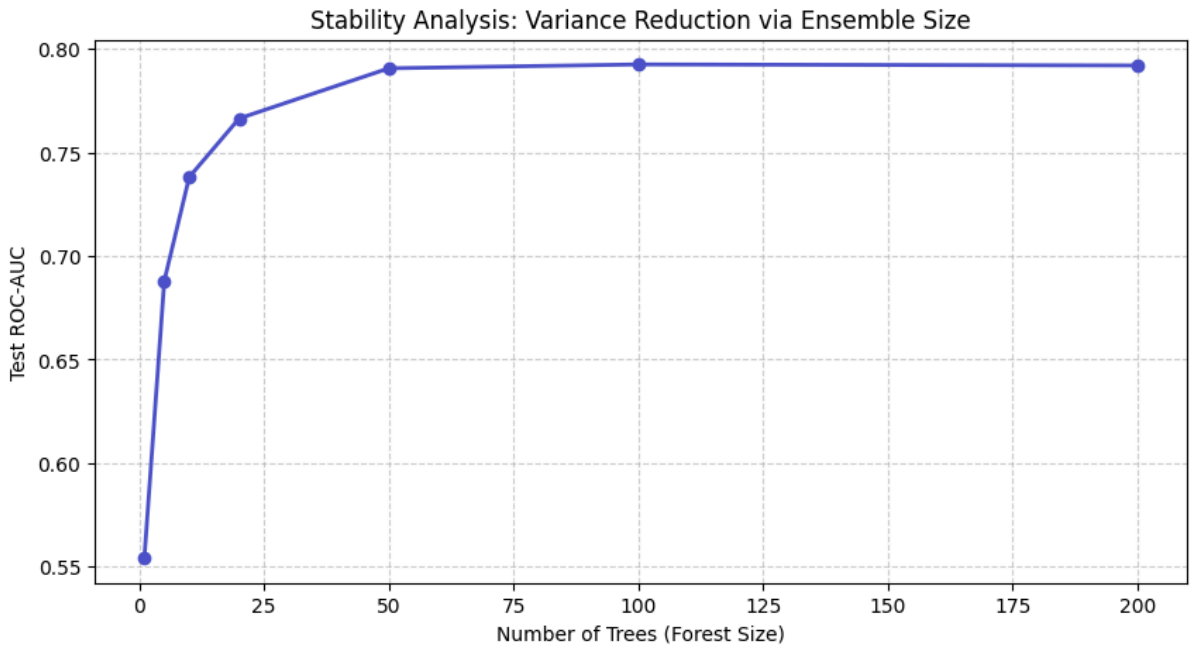
To analyse how the number of trees affects performance:

```
tree_counts = [1, 5, 10, 20, 50, 100, 200]
auc_scores = []

for n in tree_counts:
  # max_features='sqrt' triggers the Attribute Bagging mechanism
  model = RandomForestClassifier(n_estimators=n, max_features='sqrt', random_state=42)
  model.fit(X_train, y_train)
  auc = roc_auc_score(y_test, model.predict_proba(X_test)[:,1])
  auc_scores.append(auc)

plt.figure(figsize=(10, 5))
plt.plot(tree_counts, auc_scores, marker='o', color='#4e54c8', linewidth=2)
plt.title('Stability Analysis: Variance Reduction via Ensemble Size')
plt.xlabel('Number of Trees (Forest Size)')
plt.ylabel('Test ROC-AUC')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

This experiment shows that performance improves initially and stabilizes as the number of trees increases, demonstrating variance reduction.



10. Feature Importance

Random Forests provide a measure of feature importance based on how much each feature reduces impurity.

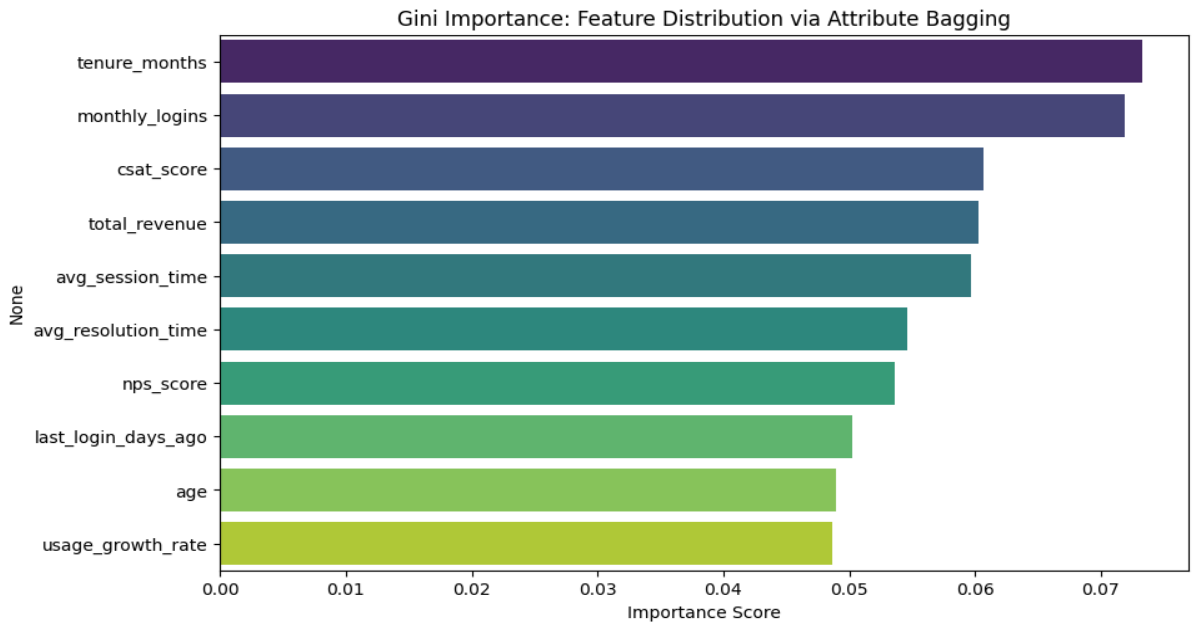
```
import seaborn as sns
```

```
importances = rf_final.feature_importances_
indices = np.argsort(importances)[::-1]



plt.figure(figsize=(10, 6))
sns.barplot(x=importances[indices][:10], y=X.columns[indices][:10], palette='viridis')
plt.title("Gini Importance: Feature Distribution via Attribute Bagging")
plt.xlabel('Importance Score')
plt.show()
```

This helps identify key drivers of churn, such as low satisfaction scores or high support ticket counts.



11. Results and Analysis

Comparing a single Decision Tree with a Random Forest:

Metric	Single Decision Tree	Random Forest (Attribute Bagging)
Accuracy	82.20%	**89.95%**
ROC-AUC	0.564	**0.794**
Recall (Churners)	0.35	**0.68**

The most significant improvement is in recall, which is crucial in churn prediction because failing to identify a churner can result in lost revenue.

12. Advantages and Limitations

12.1 Advantages:

- Lessens overfitting
- Deals with complicated relationships
- Gives information about feature importance
- Works well with different kinds of data

12.2 Limitations:

- Not as easy to understand as single models
- Costly in terms of computing
- Needs to have its parameters set right

12. Ethical Considerations

Machine learning models can show biases that are already in the data. For instance, if some groups of customers have always gotten worse service, the model might unfairly predict that those groups will leave more often.

To fix this:

- Check out the audit features closely
- Use metrics of fairness
- Use SHAP and other tools that help you understand things.

16. Conclusion

Random Forests are a strong way to fix the problem of Decision Trees overfitting. They make a strong and diverse ensemble by putting together several trees and adding randomness through attribute bagging.

This tutorial showed how attribute bagging makes trees less correlated, which makes them better at generalizing and predicting. In terms of predicting churn, this means that businesses can find at-risk customers more accurately and make better decisions.

References

1. Amit, Y. and Geman, D. (1997) ‘Shape quantization and recognition with randomized trees’, *Neural Computation*, 9(7), pp. 1545–1588.
2. Breiman, L. (1996) ‘Bagging predictors’, *Machine Learning*, 24(2), pp. 123–140.
3. Breiman, L. (2001) ‘Random forests’, *Machine Learning*, 45(1), pp. 5–32.
4. Ho, T. K. (1998) ‘The random subspace method for constructing decision forests’, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(8), pp. 832–844.
5. James, G., Witten, D., Hastie, T. and Tibshirani, R. (2013) *An Introduction to Statistical Learning*. New York: Springer.